

The OpenMP Common Core: Most OpenMP programs only use these 19 items

OpenMP pragma, function, or clause	Concepts
#pragma omp parallel	Parallel region, teams of threads, structured block, interleaved execution across threads
int omp_get_thread_num() int omp_get_num_threads()	Create threads with a parallel region and split up the work using the number of threads and thread ID
double omp_get_wtime()	Speedup and Amdahl's law. False Sharing and other performance issues
setenv OMP_NUM_THREADS N	Internal control variables. Setting the default number of threads with an environment variable
#pragma omp barrier #pragma omp critical	Synchronization and race conditions. Revisit interleaved execution.
#pragma omp for #pragma omp parallel for	Worksharing, parallel loops, loop carried dependencies
reduction(op:list)	Reductions of values across a team of threads
schedule(dynamic [,chunk]) schedule (static [,chunk])	Loop schedules, loop overheads and load balance
private(list), firstprivate(list), shared(list)	Data environment
nowait	Disabling implied barriers on workshare constructs, the high cost of barriers, and the flush concept (but not the flush directive)
#pragma omp single	Workshare with a single thread
#pragma omp task #pragma omp taskwait	Tasks including the data environment for tasks.



Data environment: Default storage attributes

Source: Tim Mattson, Intel

- Shared Memory programming model:
 - Most variables are shared by default
- Global variables are SHARED among threads
 - Fortran: COMMON blocks, SAVE variables, MODULE variables
 - C: File scope variables, static
 - Both: dynamically allocated memory (ALLOCATE, malloc, new)
- But not everything is shared...
 - Stack variables in subprograms(Fortran) or functions(C) called from parallel regions are PRIVATE
 - Automatic variables within a statement block are PRIVATE.



Data sharing: Examples

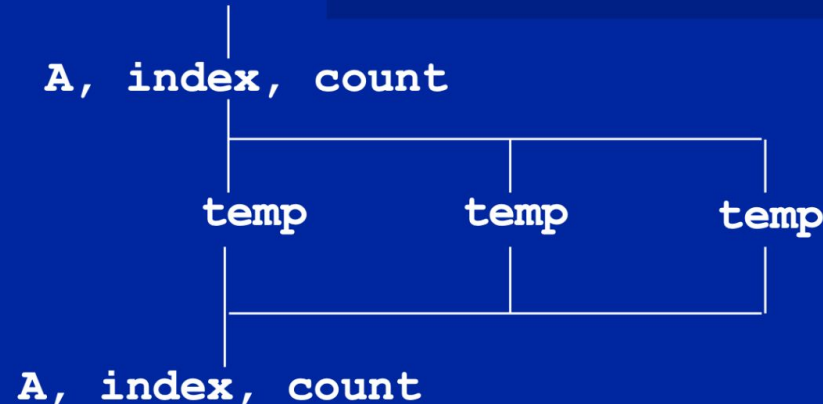
Source: Tim Mattson, Intel

```
double A[10];
int main() {
    int index[10];
    #pragma omp parallel
        work(index);
    printf("%d\n", index[0]);
}
```

A, index and count are shared by all threads.

temp is local to each thread

```
extern double A[10];
void work(int *index) {
    double temp[10];
    static int count;
    ...
}
```



Data Sharing: Default Clause

Source: Tim Mattson, Intel

- Note that the default storage attribute is **DEFAULT(SHARED)** (so no need to use it)
 - ◆ Exception: **#pragma omp task**
- To change default: **DEFAULT(PRIVATE)**
 - ◆ *each* variable in the construct is made private as if specified in a private clause
 - ◆ mostly saves typing
- **DEFAULT(NONE)**: *no* default for variables in static extent. Must list storage attribute for each variable in static extent. Good programming practice!

Only the Fortran API supports default(private).

C/C++ only has default(shared) or default(none).



The Mandelbrot Area program

Source: Tim Mattson, Intel

```
#include <omp.h>
# define NPOINTS 1000
# define MXITR 1000
void testpoint(void);
struct d_complex{
    double r;    double i;
};
struct d_complex c;
int numoutside = 0;

int main(){
    int i, j;
    double area, error, eps = 1.0e-5;
#pragma omp parallel for default(shared) private(c,eps)
    for (i=0; i<NPOINTS; i++) {
        for (j=0; j<NPOINTS; j++) {
            c.r = -2.0+2.5*(double)(i)/(double)(NPOINTS)+eps;
            c.i = 1.125*(double)(j)/(double)(NPOINTS)+eps;
            testpoint();
        }
    }
    area=2.0*2.5*1.125*(double)(NPOINTS*NPOINTS-
numoutside)/(double)(NPOINTS*NPOINTS);
    error=area/(double)NPOINTS;
}
```

```
void testpoint(void){
    struct d_complex z;
    int iter;
    double temp;

    z=c;
    for (iter=0; iter<MXITR; iter++){
        temp = (z.r*z.r)-(z.i*z.i)+c.r;
        z.i = z.r*z.i*2+c.i;
        z.r = temp;
        if ((z.r*z.r+z.i*z.i)>4.0) {
            numoutside++;
            break;
        }
    }
}
```

**When I run this program, I get a
different incorrect answer each
time I run it ... there is a race
condition!!!!**



Debugging parallel programs

Source: Tim Mattson, Intel

- Find tools that work with your environment and learn to use them. A good parallel debugger can make a huge difference.
- But parallel debuggers are not portable and you will assuredly need to debug “by hand” at some point.
- There are tricks to help you. The most important is to use the `default(none)` pragma

```
#pragma omp parallel for default(none) private(c, eps)
for (i=0; i<NPOINTS; i++) {
    for (j=0; j<NPOINTS; j++) {
        c.r = -2.0+2.5*(double)(i)/(double)(NPOINTS)+eps;
        c.i = 1.125*(double)(j)/(double)(NPOINTS)+eps;
        testpoint();
    }
}
```

Using
`default(none)`
generates a
compiler
error that `j` is
unspecified.



Data sharing:

Changing storage attributes

- One can selectively change storage attributes for constructs using the following clauses*
 - **SHARED**
 - **PRIVATE**
 - **FIRSTPRIVATE**
- The final value of a private inside a parallel loop can be transmitted to the shared variable outside the loop with:
 - **LASTPRIVATE**
- The default attributes can be overridden with:
 - **DEFAULT (PRIVATE | SHARED | NONE)**
DEFAULT(PRIVATE) *is Fortran only*

All the clauses on this page apply to the OpenMP construct NOT to the entire region.

***All data clauses apply to parallel constructs and worksharing constructs except “shared” which only applies to parallel constructs.**

Source: Tim Mattson, Intel



Major OpenMP constructs we've covered so far

- To create a team of threads
 - ◆ `#pragma omp parallel`
- To share work between threads:
 - ◆ `#pragma omp for`
 - ◆ `#pragma omp single`
- To prevent conflicts (prevent races)
 - ◆ `#pragma omp critical`
 - ◆ `#pragma omp atomic`
 - ◆ `#pragma omp barrier`
 - ◆ `#pragma omp master`
- Data environment clauses
 - ◆ `private (variable_list)`
 - ◆ `firstprivate (variable_list)`
 - ◆ `lastprivate (variable_list)`
 - ◆ `reduction(+:variable_list)`

Where `variable_list` is a comma separated list of variables

Print the value of the macro
`_OPENMP`

And its value will be

`yyyyymm`

For the year and month of the spec the implementation used

